EV Challenge GoodWe

Relatório de Evolução do Projeto — EV Challenge GoodWe (Sprint 4 / RAG)

Curso: Ciência da Computação (2SEM) — FIAP 2026.2

Disciplina: Prompt and Artificial Intelligence

Professor: Jorge Luiz Gomes

Turma: 1CCPI

Data: 18/10/2026

Fonte: conteúdo do repositório (README.md, docs/, src/, evals/, INTEGRANTES.txt, git log). Itens sem dado estão marcados como [PREENCHER].

Relatório de Evolução do Projeto — EV Challenge GoodWe (Sprint 4 / RAG)

Curso: Ciência da Computação (2SEM) — FIAP 2026.2

Disciplina: Prompt and Artificial Intelligence

Professor: Jorge Luiz Gomes

Turma: 1CCPI

Fonte: todo o conteúdo abaixo já existe no repositório (`README.md`, `evals/README.md`, `docs/relatorio_rag.md`, `src/rag/*.py`, `evals/iteration_*.json`, `INTEGRANTES.txt`, `git log`). Nada foi estimado. Onde o dado não existe no repo, está marcado como `[PREENCHER]`.

---

1. Resumo da evolução (Sprints 1/2 → 3 → 4)

Sprint 1/2 — versão original (monolítica)

Conforme o histórico do repositório, o chatbot das Sprints 1/2 era entregue como um script CLI monolítico (`chatbot.py`): o histórico de mensagens vivia em uma lista Python, era enviado integralmente ao modelo a cada turno, e a resposta era consumida como string bruta, sem validação de estrutura. Não existia controle de tamanho de contexto, nem camadas de segurança, nem garantia de que a saída fosse compatível com o domínio.

[PREENCHER: detalhes específicos da Sprint 1/2 (arquivos, métricas de aprovação, prompts) não estão no repositório da Sprint 4.]

Sprint 3 — refactory para LCEL (memória e guardrails)

Na Sprint 3, o projeto foi refatorado sob o princípio de composição em cadeias LangChain (LCEL). O núcleo passou de um script único para uma estrutura desacoplada de componentes: `ChatPromptTemplate | ChatOllama | StrOutputParser/PydanticOutputParser`. A memória passou a ser gerenciada por `session_id` com `RunnableWithMessageHistory` e recorte automático via `trim_messages` (limite de 1000 tokens). Foram adicionados guardrails em duas etapas (`moderation.py` para prompt injection e `scope_validator.py` para escopo temático), validação de schema via Pydantic v2 com `@field_validator`, prompts versionados sob Context Engineering (XML Tagging) e uma suíte automatizada (`tests/`) mais um executor de avaliação (`evals/run_sprint3.py`) com 16 casos.

Fonte: commit `24393d1` (`docs/relatorio_evolucao.md` da Sprint 3, movido para `goodwe-chatbot-sprint3/` na integração).

Sprint 4 — pipeline RAG com grounding auditável

Na Sprint 4, o chatbot evolui para um sistema que **responde apenas com o que está nos manuais**, **cita a fonte** de cada afirmação e recusa quando a base não tem a resposta. Todo o fluxo de um turno converge para `src/rag/pipeline.py::responder()`, o único caminho de execução — a interface (`app/`) e a avaliação (`evals/`) consomem essa função em vez de remontar o fluxo:

```
pergunta
|
+-- 1. moderate() [Sprint 3] detecção de prompt injection
+-- 2. validate_scope() [Sprint 3] escopo temático
+-- 3. buscar_com_scores() [Sprint 4] busca vetorial + gate de contexto
+-- 4. build_rag_chain() [Sprint 4] prompt de grounding | ChatOllama
|
+--> {"resposta": str, "fontes": [...], "bloqueado": bool}
```

```
Os três primeiros passos são locais e custam milissegundos; uma pergunta bloqueada nunca chega ao LLM (não
consome token, não depende de rede e não produz resposta), e uma falha de infraestrutura (Ollama fora do ar,
base ausente) é tratada separadamente, para não se confundir com uma recusa de guardrail.

Fonte: README.md (raiz) e src/rag/pipeline.py.

---

## 2. Pipeline RAG — decisões de chunking e trade-offs

Conteúdo copiado de docs/relatorio_rag.md (etapa anterior), que por sua vez se baseia em README.md e
src/rag/chunking.py. Nada foi estimado.

### 2.1 Parâmetros de chunking
Parâmetros em src/rag/chunking.py: CHUNK_SIZE = 500 e CHUNK_OVERLAP = 100. Os parâmetros são medidos em
**caracteres**, não em tokens.
```

Medição na base atual (2 PDFs, ~2065 caracteres):

configuração	chunks	média por chunk
1000 / 200	3	740 chars
750 / 150	4	560 chars
500 / 100	**6**	**398 chars** (adotado)
300 / 60	9	227 chars

2.2 Por quê

- **1000 caracteres** (configuração anterior) misturava seções distintas no mesmo vetor: o manual GoodWe tem, em uma página, Especificações Técnicas, Segurança Elétrica e FAQ. Com 740 caracteres por chunk, uma busca por "disjuntor DR" devolvia um chunk majoritariamente sobre potência e Modbus, e o sinal relevante ficava diluído.
- **500 caracteres** cortam, na prática, na fronteira de seção: a média cai para 398 caracteres e cada chunk carrega um tema dominante. É a faixa em que um chunk continua sendo uma unidade citável — perto de 300 caracteres a especificação passa a ser separada de sua unidade ou condição.
- **Overlap de 100 (20% do chunk)** cobre o custo dessa granularidade maior: o final de um chunk reaparece no início do seguinte, então o par valor/unidade sobrevive inteiro em pelo menos um dos dois.

2.3 Ordenação dos separadores
Os separadores são tentados do corte mais estrutural para o menos; o RecursiveCharacterTextSplitter só recua para o próximo quando o trecho excede chunk_size, então os primeiros definem a granularidade real. Ordem em SEPARADORES:

1. "\n\n" — fim de parágrafo; fronteira semântica mais confiável (cada seção de um manual técnico é um parágrafo).
2. "\n" — quebra de linha solta, para blocos que separaram itens em linhas.
3. r"\n\d+\.\s+" — cabeçalho de seção numerada; fica abaixo de "\n\n" de propósito, para não fragmentar dois parágrafos que já cabem em 500 caracteres.
4. ". ", "? ", "! " — fim de sentença, com a pontuação de fim do português.
5. "; " — separador de oração coordenada, corte mais fraco que o ponto.
6. ", " e " " — últimos recursos, garantidos pelo separador vazio "" no fim.

O separador vazio final é obrigatório: sem ele, texto que não contém nenhum dos separadores acima seria devolvido inteiro, acima de chunk_size.

2.4 Limite conhecido
500/100 é um ponto de partida medido para os ~2065 caracteres desta base, não um valor definitivo para um manual de 200 páginas. Trocar o chunk_size sem um antes/depois medido é otimizar no escuro.

2.5 Base de conhecimento

Arquivo	Origem	Assunto
manual_goodwe.pdf	Fabricante (GoodWe)	Especificações técnicas, segurança elétrica, FAQ
regimento_condominio.pdf	Condomínio	Rateio de custos, regras de uso, restrições

Ambos têm 1 página; a base produz 6 chunks no total.

2.6 Grounding e citação
- O prompt vive em prompts/system_prompt_rag_v.txt e o é enviado como **mensagem de usuário**, não dentro do system prompt: conteúdo extraído de PDF é dado não confiável e precisa ficar em um nível de autoridade abaixo das regras.
- O formato de citação é [arquivo.pdf, p. N]. format_docs() rotula cada trecho no **mesmo formato** exigido para a citação, para que o modelo só copie o rótulo.
- A similaridade é de **cosseno**, em [-1, 1], calculada a partir da distância do Chroma (similarity_from_distance).
- **Limiar não calibrado.** LIMIAR_SIMILARIDADE = 0.30 (src/rag/guardrails_rag.py). Com os embeddings atuais do nomic-embed-text, as similaridades entre documentos desta base ficam na faixa 0,69–0,74, acima de 0,30: na prática, o gate só dispara quando a busca não devolve NENHUM trecho. Recalibrar exige uma base maior e casos com relevância anotada.

2.7 Trade-offs resumidos

Critério	Trade-off
Performance	O custo dominante continua sendo a latência do modelo; a sobrecarga de orquestração do LCEL é mínima por chamada.
Manutenibilidade	Estrutura modular com responsabilidades isoladas substitui um script monolítico de difícil evolução. Custo: mais arquivos e abstração para os novos integrantes.

```
| Segurança | Ordem deliberada (moderação -> escopo -> busca -> LLM) impede que uma injeção seja mascarada como fora de escopo e evita chamadas desnecessárias ao modelo. |
| Abrangência | Base pequena (2 PDFs, 1 página cada, 6 chunks): pouca granularidade para separar "a base respondeu" de "a busca não achou nada". |

---

## 3. TABELA ANTES/DEPOIS (obrigatória)

| Linha | Sprints 1/2 (versão original) | Sprint 04 (RAG avaliado) |
|---|---|---|
| Arquitetura do núcleo | Script Python monolítico manual (chatbot.py) | LCEL com RAG (src/rag/pipeline.py::responder, retriever | format_docs | prompt | ChatOllama) |
| Base de conhecimento | [PREENCHER] | 2 PDFs (1 página cada), 6 chunks, ChromaDB goodwe_kb |
| Modelo de chat | [PREENCHER] | gemma4:cloud / gpt-oss:120b-cloud / gpt-oss:20b-cloud via Ollama |
| Modelo de embeddings | [PREENCHER] | nomic-embed-text (Ollama local) |
| Qualidade do contexto recuperado | [PREENCHER] | Busca semântica com k=4; similaridade de cosseno a partir da distância do Chroma; gate de contexto com LIMIAR_SIMILARIDADE = 0,30 (não calibrado) |
| Presença de citação de fonte | [PREENCHER] | Obrigatória no formato [arquivo.pdf, p. N]; format_docs() rotula cada trecho no mesmo formato exigido |
| Scores por iteração — v3.0 (faithfulness / answer_relevancy) | [PREENCHER] | 0,6085 / 0,6107 (iteration_1.json, 9 casos) |
| Scores por iteração — v3.1 (faithfulness / answer_relevancy) | [PREENCHER] | 0,5852 / 0,5657 (iteration_2.json, 9 casos) |
| Scores por iteração — v3.1 + gpt-oss:120b-cloud (RAG e judge iguais) | [PREENCHER] | 0,5648 / 0,5142 (iteration_3.json, 9 casos) |
| Scores por iteração — v3.1 + gpt-oss:20b-cloud | [PREENCHER] | 0,5926 / 0,4878 (iteration_4.json, 9 casos) |
| Scores por iteração — v3.2 (15 casos) | [PREENCHER] | 0,7315 / 0,5090 (iteration_5.json, 15 casos; recusa correta 1,00) |
| Scores por iteração — v3.1 (15 casos, linha de base justa) | [PREENCHER] | 0,8130 / 0,5933 (iteration_v31_15casos.json, 15 casos) |
| Recusa indevida (caso conceptual_dr) | [PREENCHER] | v3.0 recusava indevidamente; v3.1 responde corretamente com citação |
| Guardrails e segurança | Verificações simples de palavras-chave | Moderação -> escopo -> busca -> LLM; gate de contexto novo (avaliar_contexto) |
| Memória por sessão | Histórico completo em memória sem limite | session_id com RunnableWithMessageHistory + trim_messages (Sprint 3) |
| Avaliação automatizada | [PREENCHER] | RAGAS (*faithfulness*, *answer_relevancy*) + rubrica manual de fallback; shim do RAGAS em evals/ragas_eval.py |
| Erros de infraestrutura | [PREENCHER] | 0 erros de infraestrutura em todas as iterações registradas |

> **Nota de comparabilidade:** as iterações 1 a 4 usam 9 casos; a iteração 5 (v3.2) e a linha de base v3.1 (15 casos) usam a base expandida de 15 casos de evals/rag_cases.json. A coluna "Ganho direto entre iterações com bases diferentes não é estritamente comparável".

---

## 4. Problemas encontrados e soluções

### 4.1 Recusa indevida (v3.0 -> v3.1)
- **Problema:** rodando o V3.0 com gemma4:cloud sobre evals/rag_cases.json, a falha dominante era **recusa indevida**, não alucinação. No caso conceptual_dr ("O que é um DR em uma instalação de recarga?"), o contexto continha "é obrigatório o uso de disjuntor DR Tipo A (30mA) para proteção contra fuga de corrente residual" e o V3.0 respondeu "Não encontrei essa informação na base de conhecimento. [...] não fornece a definição técnica do que é um DR". A resposta existe no contexto; o V3.0 proibia inventar sem dizer, em momento nenhum, que recusar também é um erro quando o contexto tem o dado.
- **Solução:** o V3.1 introduz o procedimento de decisão (A)/(B)/(C) antes da recusa, que separa "o contexto não fala do assunto" de "o contexto fala do assunto mas não usa exatamente esta palavra". Situações (A) e (B) nunca são recusadas.
- **Resultado:** o V3.1 responde corretamente a conceptual_dr com citação; happy_kwh_concept passou a ser respondido. Citações subiram de 8 para 12 (100% com fonte válida).

### 4.2 Artefato de judge no caso conceptual_demand
- **Problema:** conceptual_demand é um teste negativo: a expressão "demanda contratada" não existe em nenhum dos dois PDFs, e o comportamento correto é recusar. Nos dois prompts (v3.0 e v3.1) o modelo recusa — a resposta está correta nos dois casos. Mas o judge deu answer_relevancy = 0,81 (v3.0) e answer_relevancy = 0,00 (v3.1). Mesmo comportamento, notas diferentes: o juiz acompanhou a redação do segundo parágrafo da recusa, não o fato de ter recusado.
- **Solução/documentação:** o evals/README.md registra que esse caso sozinho responde por quase todo o delta de relevância e que uma métrica cuja nota muda com a formulação da recusa está medindo o judge, não o grounding. Por isso os dois blocos (answerable -> RAGAS; refusal_expected -> rubrica binária) nunca são
```

somados numa média única.
- **Efeito quantificado:** um único caso com amplitude 0,81 vale 0,09 na média de 9.

4.3 UnicodeEncodeError (cp1252)
- **Problema:** no Windows, o console padrão é cp1252, que não tem ok, ! nem ->. O script evals/run_rag_eval.py estouraria UnicodeEncodeError dentro de avaliar_com_ragas() (ou no except que imprime !), perdendo a iteração inteira sem mensagem útil.
- **Solução:** _configurar_stdout() força UTF-8 com errors="replace" em sys.stdout e sys.stderr antes do primeiro print, tolerando redirecionamento para arquivo.

4.4 /api/tags vs /api/show na verificação do modelo de judge
- **Problema:** _verificar_modelo_disponível() inicialmente sondava /api/tags, que é enganoso para modelos da nuvem: a lista devolve o que o Ollama já conhece localmente, e os modelos da nuvem só entram nela depois de um primeiro uso bem-sucedido. Verificado nesta máquina, /api/tags devolvia gemma4:cloud, nomic-embed-text, llama3, qwen3.5 — e gpt-oss:120b-cloud, que responde normalmente quando chamado, não estava na lista. Basear a checagem nela rejeitaria um judge perfeitamente funcional.
- **Solução:** a verificação passou a perguntar /api/show com {"model": }, que pergunta ao servidor se ele consegue resolver o modelo — distinguindo servidor fora do ar, modelo ausente e modelo utilizável.

4.5 Escopo com falsos positivos (validate_scope)
- **Problema:** validate_scope() (Sprint 3) reprova perguntas legítimas do domínio e até saudações ("Bom dia", "Obrigado!").
- **Solução/documentação:** documentado como limitação conhecida no README.md (Sprint 4); a recalibração é trabalho pendente. A ordem (escopo antes da busca) evita gastar embedding em perguntas já destinadas à recusa, mas exige que o escopo não gere falsos positivos.

---

5. Equipe e divisão de trabalho

Integrante	RM	Turma	Tarefa principal
Rafael Lins	570588	1CCPI	[PREENCHER]
Rafael Gandolfi	569036	1CCPI	[PREENCHER]
Carlos Tavares	592949	1CCPI	[PREENCHER]
Cauã Paes	569906	1CCPI	[PREENCHER]
Guilherme Miranda	573107	1CCPI	[PREENCHER]
João Pedro Soler	569725	1CCPI	[PREENCHER]

> Fonte dos nomes/RMs: INTEGRANTES.txt e README.md (raiz). A divisão de trabalho detalhada da Sprint 4 não consta no repositório; preencher com a contribuição real de cada integrante.

O que falta preencher

Os itens abaixo não puderam ser preenchidos com dados do repositório e devem ser concluídos antes de gerar o PDF final:

1. **Números das repetições por prompt:** evals/run_repetitions.py --versions 3.0 3.1 --repeticoes 3 ainda não foi executado (ou seus resultados não estão consolidados em evals/repeticoes_resumo.md). Preencher: média e desvio padrão de faithfulness e answer_relevancy por versão (v3.0 × v3.1), com 3 execuções cada, e a conclusão de se 0,6085 × 0,5657 é diferença ou ruído com os novos dados.
2. **Decisão final do `VERSAO_PROMPT_PADRAO`:** o v3.2 (prompts/system_prompt_rag_v3.2.txt) foi avaliado nos mesmos 15 casos do v3.1 e **perdeu nas duas métricas** (v3.1 = 0,8130 / 0,5933 vs v3.2 = 0,7315 / 0,5090; recusa empate em 1,00). Recomendação com base nos dados atuais: manter o v3.1 como VERSAO_PROMPT_PADRAO. Confirmar ou alterar após as repetições.
3. **Detalhes da divisão de trabalho da Sprint 4** (tabela da Seção 5): preencher com a contribuição real de cada integrante.
4. **Detalhes da versão original (Sprints 1/2)** na Tabela ANTES/DEPOIS: arquitetura, base de conhecimento, modelos, qualidade do contexto, presença de citação, scores por iteração.

Fonte primária dos dados numéricos: evals/iteration_1.json a iteration_4.json, evals/iteration_5.json e evals/iteration_v31_15casos.json. Qualquer divergência entre este relatório e esses JSONs deve ser resolvida em favor dos JSONs.